

## UNIT IV

# 4

## Database Programming using JDBC

### ***Syllabus***

*The Concept of JDBC, JDBC Driver Types & Architecture, JDBC Packages, A Brief Overview of the JDBC process, Database Connection, Connecting to non-conventional Databases Java Data Based Client / server, Basic JDBC program Concept, Statement, Result Set, Prepared Statement, Callable Statement, Executing SQL commands, Executing queries.*

### ***Contents***

- 4.1 The Concept of JDBC
- 4.2 Types of JDBC Drivers
- 4.3 JDBC Architecture
- 4.4 JDBC Packages
- 4.5 A Brief Overview of the JDBC Process
- 4.6 Executing SQL Commands
- 4.7 Database Connection
- 4.8 Basic JDBC Program Concept
- 4.9 Executing Queries
- 4.10 Statement
- 4.11 Result Set
- 4.12 Multiple Choice Questions

## 4.1 The Concept of JDBC

- ODBC stands for **Open Database Connectivity**. It is basically API i.e. Application Programming Interface.
- Using ODBC driver interface created by Microsoft, an application can access the data present in Database Management System (DBMS).
- For accessing the data from DBMS, we normally make use of Structured Query Language (SQL) statements which is popularly known as **SQL Queries**.
- JDBC stands for **Java DataBase Connectivity**. JDBC is nothing but an API (i.e. Application Programming Interface).
- It consists of various classes, interfaces, exceptions using which Java application can send SQL statements to a database. The SQL is a Structured Query Language used for accessing the database.
- JDBC is useful for both application developers and JDBC driver vendors.
- The JDBC specification is prepared by Sun Microsystems. Any third party vendor can design their own JDBC drivers using this specification. These JDBC drivers are then used by the application developers for getting connected to the database.
- JDBC is specially used for having connectivity with the RDBMS packages (such as Oracle or MYSQL) using corresponding JDBC driver.

### Review Question

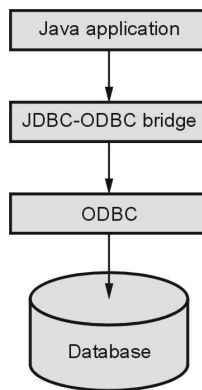
1. What is JDBC - ODBC ?

## 4.2 Types of JDBC Drivers

- There are four types of JDBC drivers and those are -
  1. **Type 1** : JDBC-ODBCBridge
  2. **Type 2** : Native-API/Partly Java Driver
  3. **Type 3** : All JAVA/ Net Protocol driver for accessing middleware server.
  4. **Type 4** : All JAVA/ Native-Protocol Pure driver
- Let us discuss them in detail -

### Type 1 : JDBC-ODBCBridge

This driver translates all the JDBC calls into ODBC (Open Database Connectivity) calls and send them to ODBC driver. Thus JDBC access is via ODBC driver. The ODBC is a generic API. In this scenario the client database code must be present on the client machine.



**Fig. 4.2.1 JDBC-ODBC bridge**

#### **Merit**

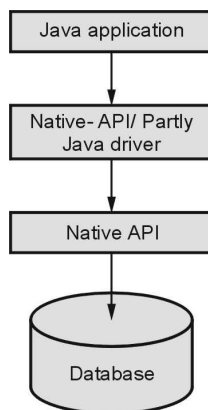
1. Using the JDBC-ODBC bridge access to any database is possible.

#### **Demerits**

1. This is slowest driver because the calls are sent to ODBC driver and then to the native database connectivity interface.
2. This type of driver is not suitable for large scale applications.
3. For using this type of driver the native database must be present on the client machine and the ODBC driver must be installed on the client's machine.

#### **Type 2 : Native-API/Partly Java Driver**

This driver translates all the JDBC calls into database-specific calls. This driver works specifically for particular database. For example MYSQL will have native MYSQL API. This type of driver directly communicates with the database server. Hence some binary code must be present on the client machine.



**Fig. 4.2.2 Type - 2 driver**

**Merit**

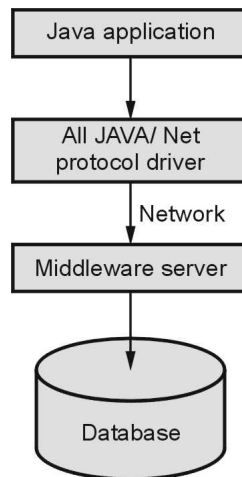
1. It gives better performance as comparison with type -1 driver because the JDBC call is directly converted to database specific call.

**Demerits**

1. The library of required databases must be loaded on the client machine.
2. This type of driver is not useful for the internet.
3. If some modifications in made in the database then the native API must also be modified because it is specific to a database.

**Type 3 : All JAVA/ Net Protocol driver for accessing middleware server**

- In this type of driver all the JDBC calls are passed through the network to the middle-ware server. The middleware server then translates the request to the database-specific native-connectivity interface and then the request is sent to the database server.
- This driver is a server-based driver. This is also known as a pure Java driver.



**Fig. 4.2.3 Type - 3 driver**

**Merits**

1. As it is server-based driver there is no need to keep library of required databases on the client machine.
2. This driver is fully written in JAVA (hence is the name all Java) and hence it is portable and can be used on internet.
3. The performance of this driver can be optimized.

4. This driver supports many advanced features such as load balancing, caching and logging.
5. For this driver it is possible to access multiple databases using one driver.

**Demerits**

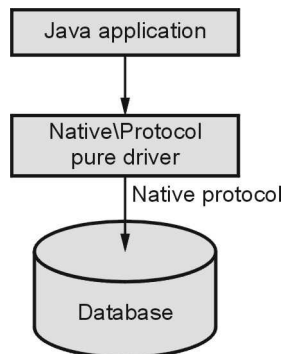
1. The middleware server application needs to be installed and maintained.
2. The record set has to traverse through the backend server.

**Type 4 : All JAVA / Native-Protocol Pure driver**

This type of driver converts the JDBC calls to network protocol used by the database directory so that the client application can directly communicate to the database server. This driver is also completely implemented in Java and hence it is referred as Pure Java driver.

**Merits**

1. As this driver is completely written in Java, it is platform independent and can be used on Internet.
2. There is no translation layer in between such as to ODBC or to native API. Neither there is a need to send the call to middle ware server. Hence the performance of this type of driver is typically good.
3. There is no need to install specific software on the client machine.
4. These drivers can be downloaded dynamically.



**Fig. 4.2.4 Type - 4 driver**

**Demerit**

1. When the type 4 driver is used then for each database a specific driver is needed.

**Review Question**

1. Explain driver types of JDBC.

### 4.3 JDBC Architecture

- JDBC API supports both two-tier and three-tier processing model.

#### 4.3.1 Two Tier Model

- In two tier model, Java application can directly communicate with database. For this communication, the JDBC driver API is required.
- The Two Tier model is represented by following Fig. 4.3.1.

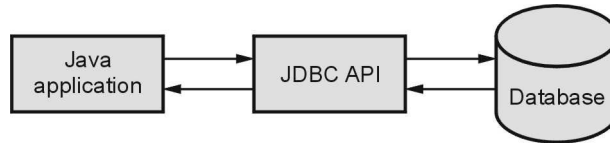


Fig. 4.3.1

#### 4.3.2 Three Tier Model

In this architecture, the HTML browser will send the command to Java Application. The Java application will then communicate with database through JDBC API. The architecture is as shown below -

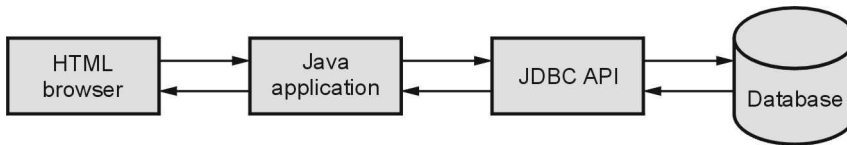


Fig. 4.3.2

#### Review Question

1. Explain JDBC architecture.

### 4.4 JDBC Packages

- The **javax.sql** is an API which contains several classes and interfaces for data source access.
- Various useful classes in **javax.sql.\*** package are :

| Class name       | Use                                                                                 |
|------------------|-------------------------------------------------------------------------------------|
| ConnectionEvent  | It is useful for getting the information of the source of connection related event. |
| PooledConnection | It allows an access to a connection pool management.                                |

|              |                                                                             |
|--------------|-----------------------------------------------------------------------------|
| RowSet       | It extends the ResultSet interface for supporting disconnected result sets. |
| XAConnection | This interface is useful for distributed transactions.                      |

- The **javax.sql.\*** is a supplementary package for **java.sql** package. Various features of this package are -
  1. The **DataSource** interface serves as an alternative to **DriverManager** class. The main advantage of using DataSource is that the applications do not need to hard code the driver class.
  2. It provides support for distributed transactions. In distributed transactions, using single transaction, data sources on multiple servers can be used by an application. The classes and interfaces used in distributed transactions are -
    - XADataSource
    - XAConnection
  3. The connection pooling is possible. The connection pooling is a mechanism by which connections can be reused. This ultimately improves the overall performance.
  4. It allows the use of RowSet.

### Mapping between JAVA and SQL

- When we wish to move the application from JAVA to SQL or vice versa then we require JDBC driver to map the data types.
- SQL data type for corresponding JAVA primitive is given by following table -

| JAVA    | SQL      |
|---------|----------|
| boolean | BIT      |
| byte    | TINYINT  |
| int     | INTEGER  |
| short   | SMALLINT |
| long    | BIGINT   |
| float   | REAL     |
| double  | DOUBLE   |

|                    |           |
|--------------------|-----------|
| String             | VARCHAR   |
| java.sql.Date      | DATE      |
| java.sql.Time      | TIME      |
| java.sql.Timestamp | TIMESTAMP |
| Byte [ ]           | BINARY    |

- Some databases treat INTEGER data type as NUMERIC. We can access the SQL datatype of each column in the database using getColumnTypes() function.

#### 4.5 A Brief Overview of the JDBC Process

- Following is a way by which JDBC works -
  - 1) First of all Java application establishes connection with the data source.
  - 2) Then Java application invokes classes and interfaces from JDBC driver for sending queries to the data source.
  - 3) The JDBC driver connects to corresponding database and retrieves the result.
  - 4) These results are based on SQL statements which are then returned to Java application.
  - 5) Java application then uses the retrieved information for further processing.

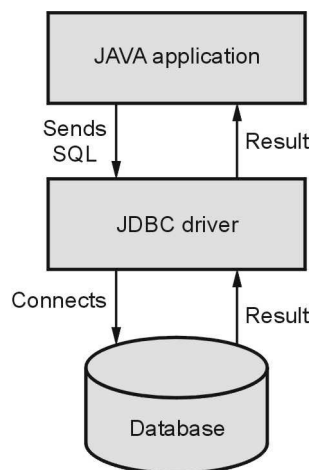


Fig. 4.5.1

#### 4.6 Executing SQL Commands

- The MYSQL is a open source database product which can be downloaded from the web site.



- After getting installed on the machine the command prompt window for MYSQL can appear. The screenshot is as follows -



- Now let us go through some MYSQL query statements which are required while handling the database.

### 1. Creating database

```
mysql> CREATE DATABASE mydb;  
Query OK, 1 row affected (0.15 sec)
```

### 2. Displaying all the databases

```
mysql> SHOW DATABASES;  
+-----+  
| Database |  
+-----+  
| mydb     |  
| mysql    |  
| students |  
| test     |  
+-----+  
4 rows in set (0.06 sec)
```

### 3. Selecting particular database

```
mysql> USE MYDB;  
Database changed
```

#### 4. Creating table

- We must create a table inside a database hence it is a common practice to use create table command after USE database command. While creating a table we must specify the table fields.

```
mysql> CREATE TABLE my_table(id INT(4),name VARCHAR(20));
Query OK, 0 rows affected (0.28 sec)
```

- Use of Primary Key :** The primary key contains the unique value. The primary key column can not contain NUL value. Each table can have only one primary key. Following is a SQL statement used to create PRIMARY KEY.

```
CREATE TABLE student_table(
roll_no INT(4) NOT NULL AUTO_INCREMENT,
name VARCHAR(50) NOT NULL,
address VARCHAR(50) NOT NULL,
PRIMARY KEY(roll_no)
);
```

- In above example the roll\_no acts as a primary key for the student\_table.

#### 5. Displaying a table

- After creating the table using SHOW command we can see all the existing tables in the current database.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_mydb |
+-----+
| my_table       |
+-----+
1 row in set (0.00 sec)
```

#### 6. Displaying the table fields

- For knowing the various fields of the table we may use following command.

```
mysql> DESCRIBE my_table;
+-----+-----+-----+-----+-----+-----+
| Field | Type   | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int(4) | YES  |     | NULL    |       |
| name  | varchar(20) | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.07 sec)
```

## 7. Inserting values into the table

- We can insert only one complete record at a time. It is as shown below -

```
mysql> INSERT INTO my_table
-> VALUES(1,'SHILPA');
Query OK, 1 row affected (0.05 sec)
```

## 8. Displaying the contents of the table

```
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name |
+-----+-----+
| 1 | SHILPA |
+-----+-----+
1 row in set (0.06 sec)
```

- We can also write SELECT statement for selecting particular row by specifying some condition such as -

```
mysql> SELECT * FROM my_table where id=1;
or
mysql> SELECT * FROM my_table where name='SHILPA';
```

- Thus we can insert the rows into the table by repeatedly giving the INSERT command.
- If we want to get the records in sorted manner then we use ORDER BY clause

```
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name |
+-----+-----+
| 1 | SHILPA |
| 2 | SUPRIYA |
| 3 | YOGESH |
| 4 | MONIKA |
+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT * FROM my_table ORDER BY name;
+-----+-----+
| id | name |
+-----+-----+
| 4 | MONIKA |
| 1 | SHILPA |
| 2 | SUPRIYA |
| 3 | YOGESH |
+-----+-----+
4 rows in set (0.00 sec)
```

## 9. Updating the record

- For updating the record in the database following command can be used -

```
mysql> UPDATE my_table
-> SET name='PRIYANKA'
-> WHERE id=4;
Query OK, 1 row affected (0.05 sec)
Rows matched: 1 Changed: 1 Warnings: 0
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name |
+-----+-----+
| 1 | SHILPA |
| 2 | SUPRIYA |
| 3 | YOGESH |
| 4 | PRIYANKA |
+-----+-----+
4 rows in set (0.00 sec)
```

## 10. Deleting record

- For deleting particular record from a database

```
mysql> DELETE FROM my_table
-> WHERE id=3;
Query OK, 1 row affected (0.04 sec)
```

- Then use SELECT statement for displaying the contents of the table we use following command

```
mysql> SELECT * FROM my_table;
+-----+-----+
| id | name |
+-----+-----+
| 1 | SHILPA |
| 2 | SUPRIYA |
| 4 | PRIYANKA |
+-----+-----+
3 rows in set (0.00 sec)
```

## 11. For deleting the table

- The table can be deleted using the command.

```
mysql> drop table my_table;
```

## 12. Inner join

- This query used to display the result from both the tables when at least one column from both the tables is matching -

For example

Consider table **Customer**

| Cust_Id | Name     | City      |
|---------|----------|-----------|
| 1       | Rahul    | Bombay    |
| 2       | Priyanka | Pune      |
| 3       | Supriya  | Bangalore |

Consider table **Order**

| Order_Id | Cust_Id | Order_Number |
|----------|---------|--------------|
| 10       | 3       | 1234         |
| 20       | 1       | 5678         |
| 30       | 1       | 8900         |

- Then we can make use of inner join query as follows -

```
SELECT Customer.Name, Customer.city, Order.Order_Number
FROM Customer
INNER JOIN Order
ON Customer.Cust_Id=Order.Order_Id
```

| Name  | City   | Order_Number |
|-------|--------|--------------|
| Rahul | Bombay | 5678         |
| Rahul | Bombay | 8900         |

### 13. Order by clause

- If we want to get the records in sorted manner then we use ORDER BY clause

```
mysql> SELECT * FROM my_table;
```

| id | name    |
|----|---------|
| 1  | SHILPA  |
| 2  | SUPRIYA |
| 3  | YOGESH  |
| 4  | MONIKA  |

```
mysql> SELECT * FROM my_table ORDER BY name;
```

| id | name    |
|----|---------|
| 4  | MONIKA  |
| 1  | SHILPA  |
| 2  | SUPRIYA |
| 3  | YOGESH  |

## 4.7 Database Connection

- **Prerequisite :** We need following things for connecting to database Using JDBC
  1. MySQL is already Installed and required database is created.
  2. JDK is installed.
  3. A database connector mysql-connector-java-X.X.XX-bin must be downloaded and present at the path.

```
C:\your_tomcat_directory\common\lib
```

4. Tomcat is running.
- For connecting java application with the mysql database, Following steps are needed to follow -
    1. **Driver class :** The driver class for the mysql database is **com.mysql.jdbc.Driver**. Here we can instantiate the object for JDBC driver by using following statement.

```
Class.forName("com.mysql.jdbc.Driver");
```

2. **Connection URL :** The connection URL for the mysql is

```
databases jdbc:mysql://localhost:3306/my_database
```

Where

- jdbc is the API,
- mysql is the database,
- localhost is the server name on which mysql is running, we may also use IP address, 3306 is the port number
- **my\_database** is the database name. (We may use any database, in such case, you need to replace the my\_database with your database name.)
- **Username :** The default username for the mysql database is **root**. If you haven't set any user name it could be blank.
- **Password :** Password is given by the user at the time of installing the mysql database.

The connection URL can be obtained using the method **getConnection**. This method is defined in class named **DriverManager**. Note that this **DriverManager** class is defined in **java.sql.\*** package. We can obtain the instance for connection using following statement

```
con = DriverManager.getConnection("jdbc:mysql://localhost:3306/my_database","","");
```

- The list of common drivers along with JDBC URL is,

### 1. ODBC Driver

#### Syntax :

```
jdbc:odbc:<data source name>
```

**Example :**

```
jdbc:odbc:MyDataSource
```

**2. MYSQL****Syntax :**

```
jdbc:mysql://[host][:port]/[database]
```

**Example :**

```
jdbc:mysql://localhost:3306/Mydatabase
```

**3. ORACLE****Syntax :**

```
jdbc:oracle:<drivertype>:<user>/<password>@<database>
```

**Example :**

```
jdbc:oracle:thin:myuser/mypassword@localhost:mydatabase
```

**4.8 Basic JDBC Program Concept**

- Following steps are used to connect JDBC to MYSQL.

**Step 1 :** Import java.sql.\* package in the JDBC program

Following line can be included in your JDBC program at the beginning.

```
import java.sql.*
```

**Step 2 :** Load JDBC Driver.

The JDBC driver for MYSQL can be loaded using following statement

```
Class.forName("com.mysql.jdbc.Driver");
```

**Step 3 :** Get connection using the Driver Manager

```
conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/my_database","root","password");
```

**Step 4 :** Create Statement

```
stmt = conn.createStatement();
```

**Step 5 :** Execute Query

```
String sql = "SELECT Roll,StudName FROM my_table";  
ResultSet rs = stmt.executeQuery(sql);
```

**Step 6 :** Display the result.

## 4.9 Executing Queries

### The executeQuery() Method

- The **executeQuery()** method is belonging to `java.sql.Statement` interface of JDBC.
- This method is used for SQL statements which retrieve some data from the database. For example is **SELECT** statement.
- This method is basically used for **SELECT** queries which fetch some data from the database. This method returns one **java.sql.ResultSet** object which contains the data returned by the query.
- This method throws the exception **SQLException**.
- The general syntax is :

```
ResultSet executeQuery(String SQL)
```

### The executeUpdate() Method

- The **executeUpdate()** method is used to update the database.
- This method makes used of all the Data Manipulation Language (DML) statements such as **INSERT**, **DELETE**, **UPDATE** or Data Definition Language (DDL) such as **CREATE**, **ALTER** and so on.
- This method returns an **int value** which represents the number of rows affected by the query. This value will be 0 for the statements which return nothing.
- This method throws the exception **SQLException**.
- The general syntax of this method is

```
int executeUpdate(String sql)
```

### Difference between executeQuery() and executeUpdate() Method

| <b>executeQuery()</b>                                                                      | <b>executeUpdate()</b>                                                                                                                                              |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| This method is used to execute SQL statements in order to retrieve data from the database. | This method is used to execute SQL statements which are intended to update or modify the database.                                                                  |
| This method returns a <b>ResultSet</b> object in which the result of the query is stored.  | This method returns integer value that represent number of rows affected by the query.                                                                              |
| This method is normally used to execute <b>SELECT</b> queries.                             | This method is used to execute non <b>SELECT</b> queries. That is-<br>DML as <b>INSERT</b> , <b>DELETE</b> , <b>UPDATE</b> or<br>DDL as <b>CREATE</b> , <b>DROP</b> |



### 4.9.1 CREATE Statement

- The database can be created using SQL query as follows -

```
CREATE DATABASE databasename;
```

- For example we can create a database named My\_database as follows -

```
CREATE DATABASE My_database;
```

- After creating the database, a table can be created within it. The query used for creating a table is

```
CREATE TABLE table_name  
(  
    column_name1 data_type(size),  
    column_name2 data_type(size),  
    column_name3 data_type(size),  
    ....  
);
```

- The *column\_name* specifies the name of the columns of the table, *data\_type* specifies the type of data the column field has. The *size* field specifies the *size* of the column in the table.
- For example

```
CREATE TABLE My_table  
(  
    Roll INT,  
    StudName VARCHAR(20)  
);
```

- With the help of Java we can execute the query using following statement.

```
Statement stat = con.createStatement();  
ResultSet rs = stat.executeQuery("CREATE TABLE MY_table  
(Roll INT, StudName VARCHAR(20))");
```

- Here is a simple Java program that illustrates how to create a table using SQL statement.

#### Java Program

```
import java.sql.*;  
public class JDBCdemo1  
{  
    public static void main(String [ ] args)  
    {  
        Connection con = null;  
        try  
        {  
            Class.forName("com.mysql.jdbc.Driver");
```

```
con = DriverManager.getConnection("jdbc:mysql://localhost/my_database","root","");
System.out.println("Connection Successful!");
Statement stat=con.createStatement();
    int result=stat.executeUpdate("CREATE TABLE My_table(Roll INT,StudName
    VARCHAR(20))");
System.out.println("Table Created");
stat.close();
con.close();
}
catch (ClassNotFoundException e)
{
    System.err.println("Exception: "+e.getMessage());
}
catch (SQLException e)
{
    System.err.println("Exception: "+e.getMessage());
}
}
}
```

**Program Explanation :** In the above program,

- 1) We are first establishing the connection with the database and then using following statements particular SQL statement can be executed.

```
Statement stat=con.createStatement();
int result= stat.executeUpdate("CREATE TABLE My_table(Roll INT, StudName
VARCHAR(20))");
```

- 2) First of all the object or the instance of **Statement** named *stat* is created.
- 3) Then the **executeUpdate()** method of that object is written in which the complete SQL query is passed. Thus we can create the table in our database.

### Output

```
c:\test>javac JDBCdemo1.java
c:\test>java JDBCdemo1
Connection Successful!
Table Created
```

## 4.9.2 SELECT Statement

- Using the **select** statement we can get the result from the database the result is obtained in the instance of **ResultSet**. The **executeQuery** function returns the **ResultSet** Object. The syntax of **executeQuery** is

```
ResultSet executeQuery(Strin SQL);
```

- The **ResultSet** is basically an interface which provides access to the table of data generated by executing a statement. The table rows are retrieved in sequence using the name of the column field.
- Following Java code is used for getting the result and displaying them,

**Prerequisite** - i) Database is created                      ii) The table is stored in this database  
iii) Some record is inserted in the table.

## JDBCDemo3.java

```
import java.sql.*;

public class JDBCdemo3 {

    public static void main(String [ ] args) {

        Connection con = null;

        try {

            Class.forName("com.mysql.jdbc.Driver");
            con=DriverManager.getConnection("jdbc:mysql://localhost/my_database","root","");
            Statement stat=con.createStatement();

            // Using SELECT Query
            String sql="SELECT * FROM My_table";
            ResultSet rs=stat.executeQuery(sql);

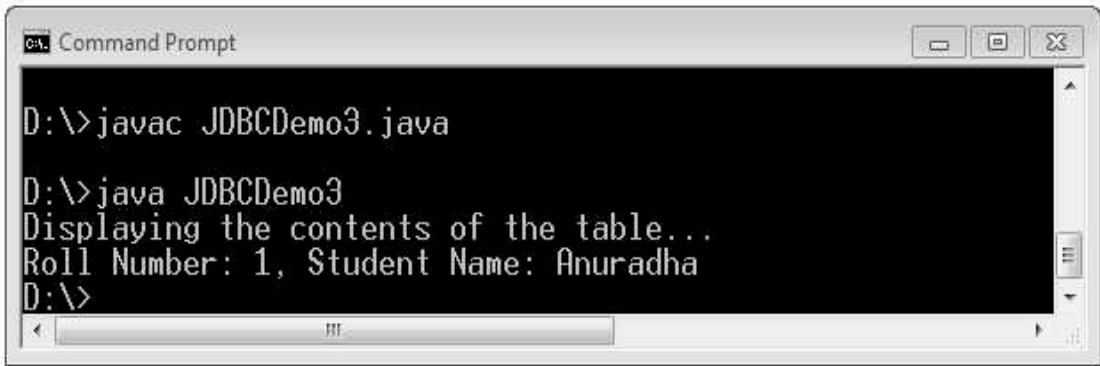

            System.out.println("Displaying the contents of the table...");
            while(rs.next())
            {
                int RollNo  = rs.getInt("Roll");
                String Name = rs.getString("StudName");

                //Display values
                System.out.print("Roll Number: " + RollNo);
                System.out.print(", Student Name: " + Name);
            }
            rs.close();
            stat.close();
            con.close();

        }
        catch (ClassNotFoundException e) {
            System.err.println("Exception: "+e.getMessage());
        }
        catch (SQLException e) {
            System.err.println("Exception: "+e.getMessage());
        }

    }

}
```

**Output**

```
Command Prompt

D:\>javac JDBCdemo3.java

D:\>java JDBCdemo3
Displaying the contents of the table...
Roll Number: 1, Student Name: Anuradha
D:\>
```

**4.9.3 UPDATE Statement**

- For updating data in the database the UPDATE query must be executed. The syntax of UPDATE command is

```
UPDATE table_name
SET column1 = value1, column2 = value2..., columnN = valueN
WHERE [condition];
```

- The **condition** can be combined using AND and OR operators.
- The steps to update the data in the database are as follows -

**Step 1 :** Create the instance for Connection class using **DriverManager's getConnection** method.

**Step 2 :** Then invoke the createStatement method in order to create the object for **Statement** class.

**Step 3 :** With the help of Statement class object the method **executeUpdate** will be invoked. The SQL query for UPDATE record can be passed as a parameter to **executeUpdate** method. The general syntax for executeUpdate is

```
public int executeUpdate(java.lang.String sql)
```

The **sql** represents the Query string using INSERT, DELETE or UPDATE.

The return value is integer which indicates number of rows affected.

Following is a Java program that updates the record stored in the table of a database.

**JDBCdemo4.java**

```
import java.sql.*;

public class JDBCdemo4 {
    public static void main(String [ ] args) {
```

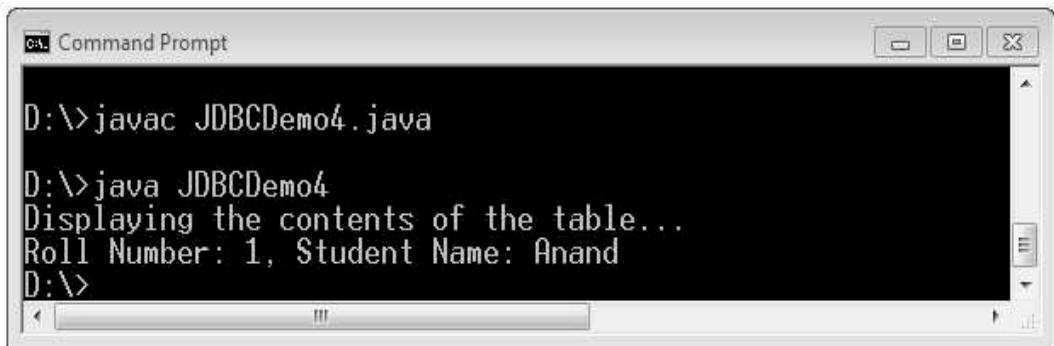
```
Connection con = null;
try {
    Class.forName("com.mysql.jdbc.Driver");
    con = DriverManager.getConnection("jdbc:mysql://localhost/my_database","root","");
    Statement stat=con.createStatement();

    String sql="UPDATE My_table " + "SET StudName='Anand' WHERE Roll=1";
    stat.executeUpdate(sql);

    sql="SELECT * FROM My_table";
    ResultSet rs=stat.executeQuery(sql);

    System.out.println("Displaying the contents of the table...");
    while(rs.next())
    {
        int RollNo = rs.getInt("Roll");
        String Name = rs.getString("StudName");
        //Display values
        System.out.print("Roll Number: " + RollNo);
        System.out.print(", Student Name: " + Name);
    }
    rs.close();
    stat.close();
    con.close();
}
catch (ClassNotFoundException e) {
    System.err.println("Exception: "+e.getMessage());
}
catch (SQLException e) {
    System.err.println("Exception: "+e.getMessage());
}
}
```

### Output



```
C:\> javac JDBCdemo4.java
C:\> java JDBCdemo4
Displaying the contents of the table...
Roll Number: 1, Student Name: Anand
C:\>
```

**Example 4.9.1** Consider bank table with attributes AccountNo, CustomerName, Balance, Phone and Address. Write a database application which allows insertion, updation and deletion of records in Bank table. Print values of all customers whose balance is greater than 20,000.

**Solution : Step 1 :** Create a database using suitable database such as MySQL/Microsoft Access/Oracle. The name of the database is **bankdb** and the name of the table created is **banktable**.

The sample database table will be as follows -

| AccountNo | CustomerName | Balance | Phone      | Address |
|-----------|--------------|---------|------------|---------|
| 11        | AAA          | 10000   | 1111111111 | Pune    |
| 22        | BBB          | 20000   | 2222222222 | Chennai |
| 44        | DDD          | 30000   | 4444444444 | Delhi   |
| 55        | EEE          | 40000   | 5555555555 | Pune    |

**Step 2 :** The java program for handling this database for given operations can be written as follows. Here we have taken the file name as **test.java**.

```
import java.sql.*;
public class test {
    public static void main(String [] args) {
        Connection con = null;
        int result;
        ResultSet rs = null;
        Statement stat=null;
        try {
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
            con = DriverManager.getConnection("jdbc:odbc:bankdb", " ", " ");
            stat = con.createStatement();
            result = stat.executeUpdate("INSERT INTO banktable" +
            "(AccountNo,CustomerName,Balance,Phone,Address)" +
            "VALUES(55,'EEE',40000,5555555555,'Pune')");
            System.out.println("Values inserted in table!");
            result = stat.executeUpdate("DELETE FROM banktable WHERE AccountNo=33");
            System.out.println("Values deleted from table!");
            result = stat.executeUpdate("UPDATE banktable set AccountNo=100 WHERE
            AccountNo=11");
            System.out.println("Values updated from table and updated record is as follows....");
```

```
rs = stat.executeQuery("SELECT * FROM banktable WHERE AccountNo=100");

while (rs.next())
{
    System.out.println("AccountNo: " + rs.getObject(1).toString());
    System.out.println("CustomerName: " + rs.getObject(2).toString());
    System.out.println("Balance: " + rs.getObject(3).toString());
    System.out.println("Phone: " + rs.getObject(4).toString());
    System.out.println("Address: " + rs.getObject(5).toString());
}
rs = null;
System.out.println("Following records having salary>20000...");
rs = stat.executeQuery("SELECT * FROM banktable WHERE Balance>20000");
System.out.println("AccountNo   Name   Balance");
while (rs.next())
{
    if (rs != null)
        System.out.println(rs.getObject(1).toString() + "      " + rs.getObject(2).toString() +
            "      " + rs.getObject(3).toString());
}
}
catch (ClassNotFoundException e)
{
    System.err.println("Exception: " + e.getMessage());
}
catch (SQLException e)
{
    System.err.println("Exception: " + e.getMessage());
}
finally
{
    try
    {
        if(rs!=null)
        {
            rs.close();
            rs=null;
        }

        if(stat!=null)
        {
            stat.close();
            stat=null;
        }

        if (con != null)
        {
            con.close();
        }
    }
}
```

```
        con = null;
    }

    }catch (SQLException e) { }
}
}
```

#### 4.10 Statement

- A **Statement** object is used for executing a static SQL statement and obtaining the results produced by it.
- The **Statement** interface can not accept parameters.

##### Creation of Statement Object

- The statement object can be created using the **createStatement()** method. Following is a illustrative Java code.

```
try {
    Class.forName("com.mysql.jdbc.Driver");
    con = DriverManager.getConnection("jdbc:mysql://localhost/my_database","root","");
    Statement stat=con.createStatement();
    ...
}
catch(SQLException ex) {
    ...
}
```

- After creating the statement object one of the following three methods can be invoked.

1. **execute** : It Returns true if a **ResultSet** object can be retrieved; otherwise, it returns false. Use this method to execute SQL DDL statement.

The general syntax is

```
bool execute(String Q)
```

2. **executeUpdate** : It returns the number of rows affected by the execution of the SQL statement. Use this method to execute SQL statements using the INSERT, UPDATE or DELETE statement. The general syntax is

```
int executeUpdate(String Q)
```

3. **executeQuery** : It returns a **ResultSet** object. Use this method when you expect to get a result set. Normally this statement is used with a SELECT statement. The general syntax is

```
ResultSet executeQuery(String Q)
```



## Closing Statement Object

Using a call to **close** method the Statement object can be closed. For instance

```
Stat.close();
```

## Types of Statements :

There are two types of statements.

1. Prepared statement
2. Callable statement

Let us discuss them in detail.

### 4.10.1 Prepared Statement

- The **java.sql.PreparedStatement** interface object represents a precompiled SQL statement.
- This interface is used to efficiently execute SQL statements multiple times. That is when we want to insert a record in a table by putting different values at runtime.
- This statement is derived from the Statement class.
- The PreparedStatement interface can be created by calling **prepareStatement()** method.
- The **prepareStatement()** is available in **java.sql.Connection** interface.
- The **prepareStatement()** method takes SQL statement in java format.

```
prepareStatement("insert into student values(?,?)").
```

where each ? represents the column index number in the table. If table **student** has **rollnumber** and **name** columns, then ? refers to **rollnumber**, ? refers to **name**.

- After that we need to set the value to each ? by using the setter method from PreparedStatement interface as follows :

```
setXXX(ColumnIndex,value)
```

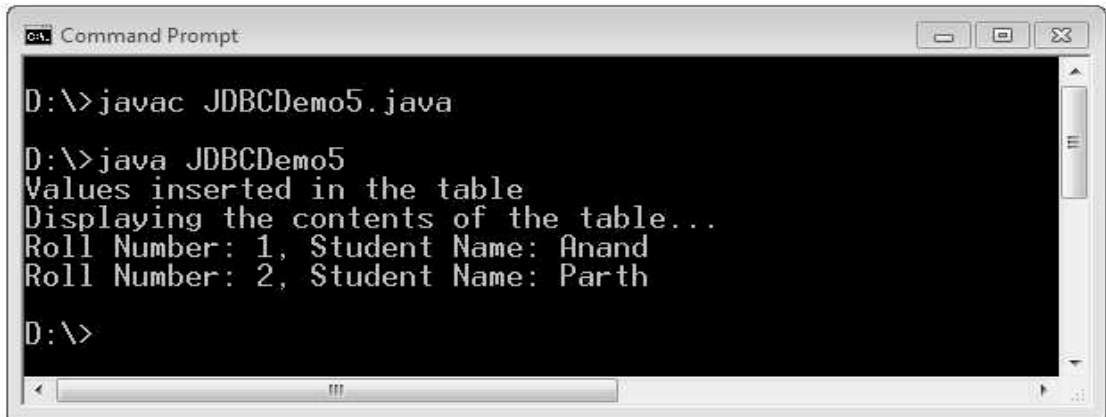
- Various setter methods are

| SQL datatype          | Method used |
|-----------------------|-------------|
| char/varchar/varchar2 | setString() |
| int/number            | setInt()    |
| float/number          | setFloat()  |
| double/Float          | setDouble() |
| long/int              | setLong()   |
| int/short             | setShort()  |
| time                  | setTime()   |
| datetime/date         | setDate()   |

Following is a Java Program that makes use of Prepared Statement for inserting data in the table.

**JDBCDemo5.java**

```
import java.sql.*;
public class JDBCDemo5 {
    public static void main(String [ ] args) {
        Connection con = null;
        try {
            int rollnum=2;
            String name="Parth";
            Class.forName("com.mysql.jdbc.Driver");
            con = DriverManager.getConnection("jdbc:mysql://localhost/my_database","root","");
            PreparedStatement ps=con.prepareStatement("INSERT INTO My_table
            VALUES(?,?)");
            ps.setInt(1,rollnum);
            ps.setString(2,name);
            int result=ps.executeUpdate();
            if(result!=0)
                System.out.println("Values inserted in the table");
            else
                System.out.println("Values are not inserted in the table");
            String sql="SELECT * FROM My_table";
            ps=con.prepareStatement(sql);
            ResultSet rs=ps.executeQuery();
            System.out.println("Displaying the contents of the table...");
            while(rs.next()) {
                int RollNo = rs.getInt("Roll");
                String Name = rs.getString("StudName");
                //Display values
                System.out.print("Roll Number: " + RollNo);
                System.out.println(", Student Name: " + Name);
            }
            rs.close();
            ps.close();
            con.close();
        }
        catch (ClassNotFoundException e) {
            System.err.println("Exception: "+e.getMessage());
        }
        catch (SQLException e) {
            System.err.println("Exception: "+e.getMessage());
        }
    }
}
```

**Output**

```
Command Prompt

D:\>javac JDBCdemo5.java

D:\>java JDBCdemo5
Values inserted in the table
Displaying the contents of the table...
Roll Number: 1, Student Name: Anand
Roll Number: 2, Student Name: Parth

D:\>
```

The callable statement is used when we want to access the **database stored procedures**. The stored procedure is basically a block of code which is identified by unique name. Let us first understand how to create procedure.

**Creating Procedure**

- The procedure can be created using following syntax.

```
DELIMITER //
CREATE PROCEDURE procedureName(parameters_list)
BEGIN
SQL Statements to be executed
END //
```

**Calling Procedure**

When calling the stored procedure, the **CallableStatement** object is used. For this object three types of parameters are used.

| Parameter | Description                                                                                                                                           |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| IN        | A parameter whose value is unknown when the SQL statement is created. Then the values can be associated with IN parameters with the setXXX() methods. |
| OUT       | A parameter whose value is supplied by the SQL statement it returns. These values are obtained in OUT parameters with the getXXX() methods.           |
| INOUT     | A parameter that supplies input as well as accepts output parameter requires a call to the appropriate setXXX method.                                 |

You can create an instance of a **CallableStatement** by calling the **prepareCall()** method on a connection object. Here is an example :

```
CallableStatement callableStatement = connection.prepareCall("{call myprocedure(?, ?)}");
```

### Setting the values to parameters

We can set the values to the paramters using setXX method. For example :

```
connection.prepareCall("{call myprocdeures(?, ?)}");
callableStatement.setInt(1, 10);
callableStatement.setString (2, "AAA");
```

### Executing the CallableStatement

Once you have set the parameter values you need to set, you are ready to execute the CallableStatement. Here is how that is done :

```
ResultSet result = callableStatement.executeQuery();
```

Then using the instance of **ResultSet** we can display the result.

Following is a simple Java application program (illustrated in stepwise manner) that makes use of callable statement.

**Step 1 :** Create a database in MySQL with the table. Insert some values inside the table. The contents of the table are represented by following Screenshot.

```

C:\xampp\mysql\bin>mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 87
Server version: 10.1.8-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2015, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

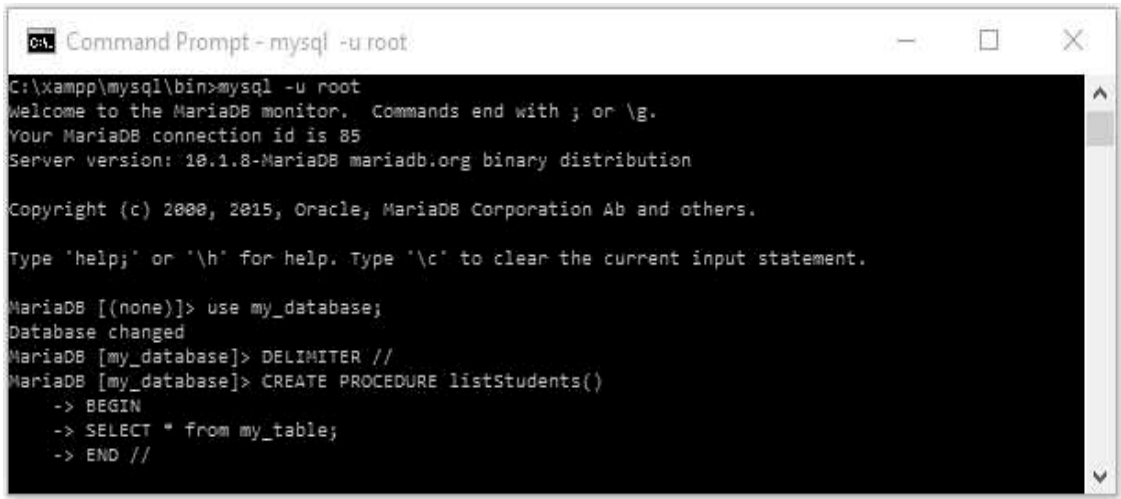
MariaDB [(none)]> USE my_database;
Database changed
MariaDB [my_database]> SELECT * from my_table;
+----+-----+
| Roll | StudName |
+----+-----+
| 1    | Anand    |
| 2    | Parth    |
| 3    | Anuja    |
| 4    | Swapnali |
+----+-----+
4 rows in set (0.00 sec)

MariaDB [my_database]>
  
```

- The sample table **my\_table** is as follows -

| Roll | StudName |
|------|----------|
| 1    | Anand    |
| 2    | Parth    |
| 3    | Anuja    |
| 4    | Swapnali |

**Step 2 :** Then Create a procedure as follows -



```
Command Prompt - mysql -u root
C:\xampp\mysql\bin>mysql -u root
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 85
Server version: 10.1.8-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2015, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> use my_database;
Database changed
MariaDB [my_database]> DELIMITER //
MariaDB [my_database]> CREATE PROCEDURE listStudents()
-> BEGIN
-> SELECT * from my_table;
-> END //
```

The procedure is as follows -

```
>use my_database
> DELIMITER //
>CREATE PROCEDUTR listStudents()
-> BEGIN
-> SELECT * FROM my_table
-> END //
```

### Step 3 :

- Before you start writing the application make sure that the driver for MySQL is present in the /tomcat/lib directory.  
It should be **mysql-connector-java-5.xx.xx-bin.jar** file.
- If it is not present then it can be downloaded from the site  
<https://dev.mysql.com/downloads/connector>
- Go for **Platform independent option** and select Zip or Tar file option depending upon your need. Then get is downloaded and save it to tomcat's **lib** directory.
- The Java code that makes use of Callable Statement for invoking the created procedure is as given below.

### JDBCDemo6.java

```
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
```

```
public class JDBCdemo6 {  
    public static void main(String[] args)  
    {  
        Connection con=null;  
        try {  
            //Getting the driver for MyQL  
            Class.forName("com.mysql.jdbc.Driver");  
            con = DriverManager.getConnection("jdbc:mysql://localhost/my_database","root","");  
            //Creating instance of CallableStatement using prepare call  
            //The procedure listStudents() created in Step 2 is invoked here  
            CallableStatement cs=con.prepareCall("CALL listStudents()");  
            ResultSet rs=cs.executeQuery();  
            while(rs.next()){  
                System.out.println(rs.getInt(1)+"\t"+rs.getString(2));  
            }  
        }catch(Exception e){  
            e.printStackTrace();  
        }  
        finally {  
            try {  
                con.close();  
            }catch(SQLException e){  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

#### 4.11 Result Set

- The ResultSet interface is an important interface which is used to access the database table with general width and unknown length.
- The table rows are retrieved in sequence using **ResultSet** object. Within a row its column values can be accessed in any order.
- A ResultSet maintains a cursor pointing to its current row of data. Initially the cursor is positioned before the first row. The 'next' method moves the cursor to the next row.
- The ResultSet object can be created using **executeQuery()** method. For example

```
Statement statement = connection.createStatement();  
ResultSet result = statement.executeQuery("select * from my_table");
```

### 4.11.1 Navigating Methods

- Various commonly used navigating methods for `ResultSet` are as given in the following table.

| Sr. No. | Methods                                              | Description                                                                                                   |
|---------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 1.      | <b>public boolean first() throws SQLException</b>    | Moves the cursor to the first row.                                                                            |
| 2.      | <b>public void last() throws SQLException</b>        | Moves the cursor to the last row.                                                                             |
| 3.      | <b>public boolean previous() throws SQLException</b> | Moves the cursor to the previous row.<br>This method returns false if the previous row is off the result set. |
| 4.      | <b>public boolean next() throws SQLException</b>     | Moves the cursor to the next row. This method returns false if there are no more rows in the result set.      |
| 5.      | <b>public int getRow() throws SQLException</b>       | Returns the row number that the cursor is pointing to.                                                        |

### 4.11.2 Reading the Result using ResultSet

- There are various methods using which the data can be retrieved using the `ResultSet` object. These methods can be used with either column Name or with column Index. Few commonly used methods are

| Sr. No. | Methods                                 | Description                                                                |
|---------|-----------------------------------------|----------------------------------------------------------------------------|
| 1.      | <b>public int getInt(int index)</b>     | Returns the integer value in the current row specified by the index.       |
| 2.      | <b>public int getInt(String Name)</b>   | Returns the integer value in the current row specified by the column Name. |
| 3.      | <b>public Date getDate(int index)</b>   | Returns the Date value in the current row specified by index.              |
| 4.      | <b>Public Date getDate(String Name)</b> | Returns the Date value in the current row specified by Name of the column. |

- Similar to **getInt** there are methods such as **getString**, **getBoolean**, **getByte**, **getFloat**, **getDouble** and so on.

### 4.11.3 Updating ResultSets

- There are various methods for updating the ResultSets denoted by **updateXXX()**. Just similar to **getXX** method the update method makes use of Column Index and Column Name. These are as given below -

| Sr. No. | Methods                                                             | Description                                                                           |
|---------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 1.      | Public void updateString(int <i>index</i> , String <i>new_val</i> ) | Updates the string by <i>new_val</i> which is specified by <i>index</i> .             |
| 2.      | Public void updateString(int <i>Name</i> , String <i>new_val</i> )  | Updates the string by <i>new_val</i> which is specified by <i>Name</i> of the column. |

- The row values can be updated using the methods as given below.

| Sr. No. | Methods                 | Description                            |
|---------|-------------------------|----------------------------------------|
| 1.      | public void updateRow() | Updates current row from database.     |
| 2.      | public void deleteRow() | Deletes the current row from database. |
| 3.      | Public void insertRow() | Inserts the row in the database.       |

### 4.12 Multiple Choice Questions

**Q.1** JDBC stands for \_\_\_\_\_.

- ☐ a Java Database Connectivity      ☐ b Java Database Control  
☐ c Java Database Components      ☐ d None of these

**Q.2** Which statements about JDBC are true ?

- ☐ a JDBC is an API to connect to relational-, object and XML data sources.  
☐ b JDBC stands for Java DataBase connectivity.  
☐ c JDBC is an API to access relational databases, spreadsheets and flat files.  
☐ d JDBC is an API to bridge the object-relational mismatch between OO programs and relational databases.

**Q.3** Which packages contain the JDBC classes ?

- ☐ a java.jdbc and javax.jdbc      ☐ b java.jdbc and java.jdbc.sql  
☐ c java.sql and javax.sql      ☐ d java.rdb and javax.rdb

**Q.4** JDBC technology-based drivers generally fit into how many categories ?

- ☐ a 4      ☐ b 3  
☐ c 2      ☐ d 5